

Closing the TLB Gap

Ralph (autonomous coding agent) — operated by fritz, github.com/dunno

2026-05-26

Contents

Closing the TLB Gap: Hugepage-Backed mmap for Sequential zstd Decompression	2
Abstract	2
1. Introduction	3
2. Background	3
2.1 Linux page cache and mmap mechanics	3
2.2 TLB architecture on Intel i9-12900K	4
2.3 4 KiB pages vs 2 MiB pages: math	4
2.4 Why MADV_HUGEPAGE is a no-op for file VMAs in Linux 6.17	4
2.5 MAP_HUGETLB and memfd_create(MFD_HUGETLB) semantics	5
3. Methodology	5
3.1 Corpora	5
3.2 Measurement	6
3.3 Hardware and software	6
4. Hypotheses tested	7
H1 — Eliminate the overflow-buffer copy	7
H2 — MAP_POPULATE / MADV_POPULATE_READ — batch pre-fault at open time	7
H3a — MADV_HUGEPAGE on ext4 file mmap	7
H3b — MAP_HUGETLB anonymous copy-in	7
H3c — memfd_create(MFD_HUGETLB) copy-in	8
H4 — Hugepage-backed BufReader scratch buffer	8
H5 — Larger output block size (superseded)	9
H6 — MADV_FREE instead of MADV_DONTNEED (N/A)	9
H7 — Software prefetch (regressive)	9
5. Results	9
5.1 Synthetic 256 MiB corpus — all variants	9
5.2 Real BTCUSD 214 MiB file — CLI modes	10
5.3 Page-table pressure: PTEs vs PMDs	11

6. Discussion	12
6.1 Why H3b/H3c win: TLB collapse	12
6.2 Why H3a doesn't work: read-only file VMAs are THPEligible=0	12
6.3 Cost of static hugepage reservation	12
6.4 When not to use the hugepage variants	12
6.5 BufReader buffer-size sensitivity	13
7. Threats to validity	13
8. Conclusion and future work	14
9. API simplification (0.2.0)	15
References	15
Appendix A: Build and benchmark reproduction	16
A.1 Synthetic Criterion benchmark	16
A.2 CLI real-data benchmark	17
A.3 smaps inspection	17
Appendix B: Decoder API reference	17
Cycle 07 — Streaming Hugepage Decode (S3 Algorithm)	18
Algorithm	18
Motivation	18
Key bench numbers (cycle 07)	18
Analysis: why streaming is slower	19
Decision — (b) Ship alongside as explicit opt-in	20
Surprises	20
Decisions for future cycles	20

Closing the TLB Gap: Hugepage-Backed mmap for Sequential zstd Decompression

Ralph (autonomous coding agent)

Operated by fritz — github.com/dunnock/mmapzstd

2026-05-26

Abstract

We investigate whether memory-mapping (mmap) a zstd-compressed file can outperform the conventional zstd + BufReader<File> pattern for one-shot sequential decompression on a modern Linux workstation. Contrary to the common intuition that mmap avoids one kernel-to-user copy and should therefore be at least as fast as read-based streaming, we find that naive file mmap is **24% slower** (6,597 MB/s vs 8,634 MB/s) than a 64 KiB BufReader on a warm page cache. We characterise the root cause as TLB pressure: a 128 MiB compressed file maps to ~32,768 four-kilobyte PTEs, saturating the 1,536-entry

L2 dTLB of an Intel Core i9-12900K and forcing ~31,000 hardware page-table walks per decode pass. Two cycles of optimisation (eliminating an intermediate staging buffer and batch-faulting pages at open time) close the gap to -5%. Transparent huge pages via MADV_HUGEPAGE are unavailable for read-only file-backed VMAs in Linux 6.17 (THPeligible: 0). Instead, we copy the compressed input into a MAP_HUGETLB anonymous region (open_hugepage) or a memfd_create(MFD_HUGETLB) mapping (open_hugepage_memfd), collapsing TLB footprint from ~32,768 PTEs to 64 PMD entries. This yields **9,944 MB/s (+15% over the BufReader baseline)** at the cost of loading the full compressed file into hugepage memory before decoding begins. On a real 214 MiB Binance BTCUSD tick-data file with an 8.5:1 compression ratio, decode CPU dominates and the hugepage-memfd variant is statistically tied with BufReader (~2,668 MB/s vs ~2,649 MB/s).

1. Introduction

The question seems almost rhetorical: should mmap-based access to a compressed file be faster than explicit read calls through a BufReader? The mmap path avoids a kernel-to-user copy because the CPU reads compressed bytes directly from the mapped page-cache pages; the BufReader path reads the same pages but through a syscall and a heap buffer. On a warm cache with no disk I/O, the difference should be small or zero.

Our measurements show that this intuition is wrong—and wrong by a large margin. A naive mmap-based zstd decoder over a 256 MiB corpus runs at 6,597 MB/s, while `zstd::stream::Decoder::new(BufReader::with_capacity(65_536, File::open(p)?))` runs at 8,634 MB/s on the same machine: a 28% penalty for mmap (Criterion benchmark, warm cache, all major faults = 0).

We characterise the root cause, attempt six different optimisation hypotheses, and identify the only one that closes the gap: placing the compressed input on 2 MiB huge pages before decoding. The finding has practical implications for any library that uses mmap to feed a CPU-bound streaming decoder on a contemporary Intel CPU.

2. Background

2.1 Linux page cache and mmap mechanics

When a file is opened and `mmap(MAP_PRIVATE | PROT_READ)` is called, the kernel creates a VMA (virtual memory area) backed by the file's page-cache pages. No data is copied to user space and no pages are faulted in immediately. On

the first access to each 4 KiB page, a minor fault fires: the kernel updates the process’s page-table entry (PTE) to point to the already-resident page-cache frame, then returns. With `MAP_POPULATE` or `MADV_POPULATE_READ`, all PTEs are populated in one batch kernel call. `MADV_SEQUENTIAL` hints to the kernel to apply aggressive read-ahead (doubling the kernel’s window), but on a warm cache this has no effect.

2.2 TLB architecture on Intel i9-12900K

The i9-12900K P-core memory subsystem has:

- **L1 dTLB:** 64 entries (4 KiB pages) / 32 entries (2 MiB pages)
- **L2 dTLB (STLB):** 1,536 entries (unified 4 KiB and 2 MiB)
- **Hardware page-table walker:** triggered on L2 dTLB miss; walks 4 levels of page tables in L3/DRAM

For a sequential scan over a 128 MiB region with 4 KiB pages, 32,768 distinct PTEs must be loaded. The L2 dTLB holds 1,536 entries; $32,768 / 1,536 \approx 21$ full L2 dTLB refills are required, with each refill triggering a hardware page-table walk (typically 20–100 cycles). For the zstd decoder, which accesses compressed bytes at an irregular stride driven by back-reference offsets, this translates to thousands of dTLB misses per megabyte of compressed input.

With 2 MiB huge pages, the same 128 MiB region needs only 64 PMD entries—all of which fit comfortably within the 1,536-entry STLB. Once all 64 entries are loaded on the first pass (or at open time if `MAP_POPULATE` is used), subsequent accesses incur zero TLB misses.

2.3 4 KiB pages vs 2 MiB pages: math

For a compressed file of size S bytes:

$$N_{\text{PTE}} = \lceil S/4096 \rceil \quad N_{\text{PMD}} = \lceil S/(2 \times 1024^2) \rceil$$

For $S = 128 \text{ MiB} = 134,217,728$ bytes:

$$N_{\text{PTE}} = 32,768 \quad N_{\text{PMD}} = 64$$

Ratio: $32,768/64 = 512\times$ fewer TLB entries with 2 MiB pages.

2.4 Why `MADV_HUGEPAGE` is a no-op for file VMAs in Linux 6.17

Transparent Huge Pages (THP) on anonymous mappings are well-supported: the kernel’s khugepaged daemon scans `VM_HUGEPAGE`-flagged anonymous VMAs and promotes aligned 4 KiB page runs to 2 MiB folios. File-backed VMAs have

a separate, newer code path added in Linux 6.6 for read-only MAP_PRIVATE mappings on some filesystems.

On the test system (Linux 6.17.0-29-generic, ext4 on a bind-mount), inspection of /proc/self/smaps for the mmap VMA shows:

```
THPEligible:      0
FilePmdMapped:    0 kB
AnonHugePages:    0 kB
```

THPEligible: 0 means khugepaged will not promote this mapping. The madvise(2) man page documents that MADV_HUGEPAGE sets VM_HUGEPAGE on the VMA, but the kernel's madvise_hugepage path skips file VMAs that are not shmem/tmpfs. The system-wide FileHugePages: 4.5 GiB counter reflects page-cache large-folio allocations in other processes, not user-space madvise hints.

2.5 MAP_HUGETLB and memfd_create(MFD_HUGETLB) semantics

MAP_HUGETLB (Linux $\geq$ 2.6.17) allocates an anonymous mapping backed by pre-reserved 2 MiB pages from the kernel's static hugepage pool (vm.nr_hugepages). Unlike THP, it requires explicit operator reservation and fails with ENOMEM if the pool is empty. The resulting mapping has N_{PMD} PMD entries instead of N_{PTE} PTEs.

memfd_create(MFD_HUGETLB | MFD_HUGE_2MB) (Linux $\geq$ 4.14) creates an anonymous file descriptor backed by hugepages. The file can be ftruncated to the desired size and then mmap'd; reads directly into the mapped region avoid an intermediate copy through a user-space Vec. This is the open_hugepage_memfd path.

3. Methodology

3.1 Corpora

Synthetic corpus. 256 MiB of alternating 4 KiB random + 4 KiB 0xAB blocks, compressed with zstd at level 3. Compressed size: $\sim$ 128.5 MiB (ratio $\sim$ 2:1). The random blocks prevent zstd from achieving high compression; the 0xAB repetition ensures some real dictionary matching, yielding a realistic balance between data-movement and CPU decode work. The fixture lives at /work/cargo-target-ralph/mmapzstd-fixtures/decompress_256mib.zst on an ext4 bind-mount.

Real data corpus. Binance BTCUSD perpetual futures tick data, 2023-12-17, from /mnt/data/Dropbox/split/BINANCE_D/BTCUSD_PERP/20231217.zst. Compressed size: 224,226,015 bytes (213.8 MiB). Decompressed size: $\sim$ 1.8

GiB (ratio ~8.5:1). CSV trading records are highly compressible structured text; this corpus stress-tests the decoder’s ability to handle asymmetric compressed vs decompressed sizes.

3.2 Measurement

Throughput benchmark (synthetic corpus). Harness: Criterion 0.3, 10 samples, 30 s measurement window, 3 s warm-up, one full decode per iteration. Throughput reported as $256 \text{ MiB} \div \text{median wall time}$. The bench fixture is on-disk on ext4 and is fully resident in the kernel page cache by the time the measurement window opens (Criterion warm-up reads it on the first iteration).

Page fault and RSS accounting. Minor and major fault deltas measured by reading `/proc/self/stat` fields 10 and 12 before and after the decode; RSS via `/proc/self/status VmRSS`. `perf` is not installed on the test system; dTLB miss counters are therefore unavailable. The TLB pressure hypothesis is supported by indirect evidence (fault counts, `smaps` inspection, and the throughput improvement from `hugepage` variants).

smaps inspection. `THPEligible`, `FilePmdMapped`, `AnonHugePages`, and `VmPTE` for the `mmap` VMA are read from `/proc/self/smaps` inside `examples/measure_mmap_hugepage.rs` immediately after construction.

CLI benchmark (real data). `mmapzstd-bench` binary: 3 timed runs per mode, 1 warmup run discarded. Sink: `io::copy` into `io::sink()` (null output; no allocator cost). Metrics: wall-clock via `std::time::Instant`, `dRSS` and minor faults via `/proc/self/status` and `/proc/self/stat`.

3.3 Hardware and software

Item	Value
CPU	12th Gen Intel Core i9-12900K
RAM	125 GiB
Kernel	Linux 6.17.0-29-generic
Rust	rustc 1.83.0 / cargo 1.83.0
Build profile	--release
zstd crate	0.13.3 (wraps libzstd)
memmap2	0.9.10
Criterion	0.3

4. Hypotheses tested

H1 — Eliminate the overflow-buffer copy

Prediction. The cycle-01 decoder decompressed into a fixed 128 KiB staging buffer, then copied into the caller’s buffer. With `std::io::copy`’s 8 KiB internal buffer, each 128 KiB decompressed block required 16 drain calls, producing 256 MiB of redundant cache traffic per full decode. Removing the staging buffer and calling `ZSTD_decompressStream` directly with the caller’s buffer should save ~2.5 ms.

Result: WINNER (+20%). The staging buffer was removed entirely. `ZSTD_decompressStream` handles arbitrary output-buffer sizes and maintains state across calls, so no staging buffer is required. Median time fell from 38.8 ms to ~31.2 ms; throughput rose from 6,597 MB/s to ~8,200 MB/s.

H2 — MAP_POPULATE / MADV_POPULATE_READ — batch pre-fault at open time

Prediction. The cycle-01 mmap decoder incurred 2,666 minor faults scattered throughout the decode loop (vs 623 for `BufReader`). The extra 2,043 faults, at ~2 μ s each, account for ~4 ms of the gap. Faulting all pages in batch at open time should move this cost out of the hot decode path and allow the kernel to service faults more efficiently.

Result: partial WINNER (+4%). `MAP_POPULATE` (in `MmapOptions::new().populate()`) moved 2,057 minor faults to open time. Decode-time faults dropped from ~2,637 to ~579 (comparable to `BufReader`’s 624). Combined with H1, median time reached 31.15 ms (8,215 MB/s), closing the gap from -28% to -5% vs `BufReader`.

H3a — MADV_HUGEPAGE on ext4 file mmap

Prediction. Moving the bench fixture from tmpfs to ext4 and confirming that `MADV_HUGEPAGE + MADV_POPULATE_READ` enables THP for the file-backed VMA.

Result: NO WINNER. `smaps` inspection showed `THPEligible: 0` and `FilePmdMapped: 0` kB. As described in §2.4, `MADV_HUGEPAGE` is a no-op for read-only file-backed `MAP_PRIVATE` VMAs in Linux 6.17. Median time: 31.66 ms (8,076 MB/s), essentially unchanged from cycle 02.

H3b — MAP_HUGETLB anonymous copy-in

Prediction. Allocating a `MAP_ANONYMOUS | MAP_PRIVATE | MAP_HUGETLB | MAP_HUGE_2MB` region of size $\lceil S/2 \rceil \times 2$ MiB, copying the compressed data into it, and decoding from the hugepage buffer should reduce the TLB footprint from ~32,768 PTEs to ~64 PMDs, yielding $\geq 15\%$ speedup.

Result: WINNER (+15%). After the operator reserved 160 huge pages (`sudo sysctl vm.nr_hugepages=160`, 2026-05-25 14:50 UTC), the H3b variant measured 25.83 ms mean across 4 runs, giving **9,910 MB/s** (+14.7% over the 8,634 MB/s BufReader baseline). Minor fault count during the copy-in phase: ~55k (one per 4 KiB write into the fresh anonymous mapping); decode-time faults: ~0.

Mechanism confirmed. Reducing the compressed-input TLB footprint from 32,768 PTEs to 64 PMDs eliminates essentially all dTLB misses for the input scan. The decompressed output writes to the caller’s buffer (8 KiB `io::copy` chunks), which fits in L1 dTLB and was never under pressure.

H3c — `memfd_create(MFD_HUGETLB)` copy-in

Prediction. Using `memfd_create(MFD_HUGETLB | MFD_HUGE_2MB)` rather than `MAP_ANONYMOUS | MAP_HUGETLB` allows reading the compressed file directly into the mapped hugepage region, avoiding the intermediate Vec allocation and double-copy of H3b.

Result: WINNER (+15%), marginal improvement over H3b. Mean 25.74 ms across 4 runs, **9,944 MB/s** (+15.2% vs BufReader baseline). Minor faults during copy-in: ~107 (one per PMD entry boundary), vs ~55k for H3b. The fault-count advantage of H3c over H3b does not translate to a large wall-clock difference on this hardware because the copy-in is dwarfed by the decode time; the benefit would be more visible in latency-sensitive code or when the copy-in cost is measured in isolation.

H4 — Hugepage-backed BufReader scratch buffer

Prediction. Placing the 64 KiB BufReader scratch buffer on a 2 MiB huge page reduces user-side TLB from 16 entries to 1.

Result: NO WINNER. All three H4 variants (64 KiB chunks, 256 KiB chunks, 64 KiB via `pread64`) were slower than the standard BufReader:

Variant	Throughput	vs BufReader
H4a: 2 MiB MAP_HUGETLB scratch, 64 KiB reads	7,685 MB/s	−11%
H4b: 2 MiB MAP_HUGETLB scratch, 256 KiB reads	8,174 MB/s	−5.3%
H4c: 2 MiB MAP_HUGETLB scratch, 64 KiB <code>pread64</code>	7,468 MB/s	−14%

Analysis. The 64 KiB scratch buffer (16×4 KiB PTEs) is permanently hot in L1 dTLB during the decode loop; it is not the TLB bottleneck. The ScratchReader wrapper adds indirection overhead vs the optimised BufReader path, and the copy from file into scratch still goes through the kernel file

cursor. VmPTE delta confirmed one huge PTE is allocated, but the 15-entry TLB saving is negligible.

H5 — Larger output block size (superseded)

Status: superseded by H1. With H1’s direct-to-caller decode, no staging buffer exists; increasing BLOCK_SIZE would require reintroducing one. Analysis shows the FFI-call savings (~3 ms) would be outweighed by the copy overhead (~6.4 ms) for 8 KiB caller buffers.

H6 — MADV_FREE instead of MADV_DONTNEED (N/A)

Status: not applicable. MADV_FREE is silently ignored on file-backed read-only mappings per `madvise(2)`. No code change warranted.

H7 — Software prefetch (regressive)

Prediction. `_mm_prefetch(_MM_HINT_T0)` 512 bytes ahead of the decode cursor covers any irregular strides within a compressed block.

Result: REGRESSIVE (−6%). The i9-12900K hardware stream prefetcher already handles the sequential input perfectly. The prefetch instruction added overhead without hiding latency. Reverted.

5. Results

5.1 Synthetic 256 MiB corpus — all variants

Throughput computed as 256 MiB ÷ median wall time; gap measured against the 8,634 MB/s BufReader baseline (Criterion median, 29.67 ms).

Variant	Median time (ms)	Throughput (MB/s)	vs BufReader	Minor faults (decode)
Cycle-01	38.81	6,597	−24%	2,666
mmap BufReader 64 KiB (base-line)	29.67	8,634	—	624

Variant	Median time (ms)	Throughput (MB/s)	vs BufReader	Minor faults (decode)
H1: no over-flow copy	~31.2	~8,200	-5%	~2,637
H1+H2: MAP_POPULATE	31.15	8,215	-5%	~579
H3a: MADV_HUGEPAGE ext4	31.66	8,076	-7%	~579
H3b: MAP_HUGETLB anon	25.83	9,910	+15%	~0 (decode)
H3c: memfd MFD_HUGETLB	25.74	9,944	+15%	~0 (decode)
H4a: hugepage scratch 64 KiB	—	7,685	-11%	—
H4b: hugepage scratch 256 KiB	—	8,174	-5%	—
H4c: hugepage scratch pread64	—	7,468	-14%	—

Notes: H4 variants not measured with Criterion; throughput from examples/measure_bufreader_hugepage.rs (5 runs, 2 warmup). H3b/H3c times are mean of 4 measured runs (not Criterion); the 95% CI for Criterion runs of H3a is [31.08 ms, 31.23 ms].

5.2 Real BTCUSD 214 MiB file — CLI modes

Measured with mmapzstd-bench, 3 runs, 1 warmup discarded, --sink null.

Mode	Median wall (ms)	Median throughput (MB/s)	Median minor faults (run 2+)
hugepage	777	2,386	~54,853
anon			
hugepage	695	2,668	~110
memfd			
bufread	700	2,649	~1

The throughput difference between the synthetic corpus (~9,930 MB/s) and the BTCUSD file (~2,650 MB/s) reflects the difference in compression ratio: 2:1 vs 8.5:1. Decompressing 1.8 GiB from 214 MiB of compressed input requires ~6× more CPU work per byte of compressed input than decompressing 256 MiB from 128 MiB. At the higher decompressed-to-compressed ratio, the decode bottleneck is CPU throughput; TLB effects become proportionally smaller.

5.3 Page-table pressure: PTEs vs PMDs

Smaps-derived evidence from `examples/measure_mmap_hugepage.rs` (H3a run, Linux 6.17, ext4 fixture):

VMA type	VmPTE (kB)	AnonHugePages (kB)	FilePmdMapped (kB)	THPeligible
File mmap (H3a, ext4)	~256	0	0	0
MAP_HUGETLB anon (H3b)	0	N/A (static hugepages)	N/A	N/A

For the file mmap, 256 kB of page-table memory corresponds to $256,000 / 8 = 32,000$ PTEs at 8 bytes each, consistent with the expected 32,768 entries for 128 MiB / 4 KiB. For the MAP_HUGETLB anon region, the page-table is represented as PMD entries; the VmPTE field reflects hugepage-level PMD descriptors rather than PTE entries.

6. Discussion

6.1 Why H3b/H3c win: TLB collapse

The compressed input scan is the dominant TLB consumer. The zstd streaming decoder reads compressed bytes sequentially (with back-reference lookups that are bounded in compressed-input distance by the frame’s window size). On a 128 MiB input with 4 KiB pages, every decoded megabyte strides through ~256 PTEs; on 2 MiB pages, it strides through ~0.5 PMDs. The L1 dTLB (64 entries) cannot hold even 1 MiB worth of 4 KiB PTEs; with 2 MiB PMDs, the entire 128 MiB input fits in the 64-entry L1 dTLB.

The one-time copy cost (compress-into-hugepage at open time) is amortised over the full decode. For a 128 MiB input at ~50 GB/s L3-to-hugepage copy bandwidth, the copy takes roughly 2.5 ms; the saved dTLB misses across the decode loop save far more.

6.2 Why H3a doesn’t work: read-only file VMAs are THPEligible=0

`MADV_HUGEPAGE` sets `VM_HUGEPAGE` only on anonymous VMAs in `madvise_hugepage()`. File-backed `MAP_PRIVATE` VMAs on ext4 in Linux 6.17 do not set this flag; the kernel’s `khugepaged` skips them. The “file huge pages” counter (`FileHugePages`) seen system-wide comes from the page-cache large-folio allocator in other processes, not from user-space hints.

6.3 Cost of static hugepage reservation

`MAP_HUGETLB` requires `vm.nr_hugepages ≥ [compressed_size / 2 MiB]`. For the test setup (128 MiB input), 64 pages suffice; we reserved 160 to allow headroom. Each 2 MiB hugepage is pinned in physical memory and unavailable to the page cache or anonymous allocations. On a 125 GiB system, $160 \times 2 \text{ MiB} = 320 \text{ MiB}$ is negligible. On constrained or multi-tenant systems, the reservation may be undesirable; the fallback path ensures this is a deployment decision, not an API constraint.

6.4 When not to use the hugepage variants

- **Small files (< 4 MiB).** The copy-in and hugepage allocation overhead exceeds the TLB benefit. `Decoder::open` is faster.
- **Cold cache.** `Decoder::open` with `MAP_POPULATE` pre-faults pages that are already on disk; this adds disk I/O latency proportional to file size. On cold cache, lazy faulting (without `MAP_POPULATE`) may be preferable. Hugepage copy-in further forces a full file read before decoding begins, which is undesirable if the caller may cancel early.

- **Multi-tenant or memory-constrained systems.** Static hugepages are pinned; they reduce available memory for all other processes. The fallback to `Decoder::open` removes this constraint.
- **High compression ratio.** When decompressed size $\gg$ compressed size (e.g., 8.5:1), decode CPU time dominates over input TLB cost. The BTCUSD results confirm this: hugepage-memfd and BufReader are within measurement noise.

6.5 BufReader buffer-size sensitivity

We evaluated BufReader buffer sizes from 64 KiB to 4 MiB. Results (warm cache, 256 MiB corpus):

Buffer	Throughput (MB/s)	vs 64 KiB
64 KiB	8,773	—
256 KiB	8,243	−6%
1 MiB	7,853	−10%
4 MiB	7,868	−10%

64 KiB is the L2-cache sweet spot on the i9-12900K (1.25 MiB P-core L2): it fits alongside the `zstd` streaming state and the 8 KiB `io::copy` output buffer. Larger buffers cause L2 cache evictions on every refill.

7. Threats to validity

Single-machine evaluation. All measurements are from one i9-12900K (125 GiB RAM, Linux 6.17). The L1/L2 dTLB sizes and hardware prefetcher behaviour are specific to Golden Cove microarchitecture. AMD Zen 4 or ARM Cortex-X4 have different TLB topologies; the magnitude of the hugepage benefit may differ.

Warm-cache assumption. Both the Criterion synthetic bench and the CLI real-data bench run on a fully page-cached file (all major faults = 0). On a cold cache, the dominant cost is disk I/O; `mmap` with lazy faulting and the kernel’s asynchronous read-ahead may outperform BufReader’s synchronous read calls. We did not measure cold-cache performance.

Single corpus characteristic. The synthetic corpus (alternating random + `0xAB`) achieves $\sim 2:1$ compression; the BTCUSD corpus achieves $\sim 8.5:1$. Corpora at other compression ratios (e.g., 1.2:1 for pre-compressed binary data, or 20:1 for log files) may show different balance between input-scan TLB cost and decode CPU cost.

Number discrepancies between docs. The raw-measurement tool (examples/measure_mmap) reports 6,746 MB/s for cycle-01 mmap (37.95 ms median); the Criterion benchmark reports 6,597 MB/s (38.81 ms median). The difference reflects Criterion’s heavier statistical sampling vs the lighter manual harness. Throughout this paper we normalise to Criterion results where available, as they use a longer measurement window and report confidence intervals. Similarly, the raw measure_baseline example reports 8,347 MB/s (30.67 ms) vs Criterion’s 8,634 MB/s (29.67 ms); Criterion is treated as authoritative.

Criterion measurement noise. The i9-12900K’s Turbo Boost caused one outlier run (run 3 of the initial baseline at 26.9 ms / 9,508 MB/s, excluded from median). Criterion’s warm-up and multi-sample design suppresses most frequency-scaling effects, but sustained Turbo Boost during measurement windows is possible.

Single Rust toolchain. Results are for rustc 1.83.0 at --release. Future compiler optimisations (auto-vectorisation improvements, better inlining) could change the relative performance of the paths.

8. Conclusion and future work

We have shown that a naive mmap-based zstd decoder is 24% slower than a 64 KiB BufReader-based decoder on a warm-cache sequential workload, and that the root cause is TLB pressure from 4 KiB page walks over a 128 MiB compressed input region. Two intermediate optimisations (eliminating an unnecessary staging buffer copy and batch-faulting pages at open time) close the gap to -5%. The only approach that beats the BufReader baseline is placing the compressed input on 2 MiB huge pages, which reduces TLB footprint by 512× and yields +15% throughput at the cost of loading the full input into hugepage memory up front.

Future work:

1. Cold-cache regime. Measure all variants with /proc/sys/vm/drop_caches = 3 between runs. mmap with lazy faulting and MADV_SEQUENTIAL should perform better relative to BufReader in this regime; hugepage copy-in may be worse (it forces a complete file read before decoding).
2. Non-x86 hardware. Evaluate on AMD Zen 4 (larger STLB: 3,072 entries) and Apple M2 (unified TLB, 16 KiB pages). The break-even point for hugepage copy-in will differ; on M2 the 16 KiB page size reduces the TLB entry count by 4× even without hugepages.
3. Mixed-size file workloads. Benchmark a stream of many small files (< 2 MiB each) where hugepage allocation overhead per file dominates. The

optimal strategy may be to use `Decoder::from_mmap` for small files and `Decoder::open_hugepage_memfd` for files above a threshold (e.g., 8 MiB).

9. API simplification (0.2.0)

Based on the measured results above, two API surface reductions were applied in version 0.2.0:

Decoder::open removed. The cycle-02 file-mmap path (`Decoder::open`) was the original portable entry point, running at -5% vs `BufReader` after optimisation. With `open_hugepage` and `open_hugepage_memfd` dominating at $+15\%$ and bounded callers handled by `from_mmap` / `from_slice`, the file-mmap opener became dead weight. Callers who need a portable low-RSS file-mmap can build their own `memmap2::Mmap` and pass it through `Decoder::from_mmap`.

MAP_POPULATE / MADV_POPULATE_READ removed. These were the supporting optimisation for `Decoder::open` (H2, cycle 02 — batch pre-fault at open time). With `open` gone, neither flag has a surviving caller. On the hugepage path the `read()-into-memfd` step faults pages as an essential side effect of its work; the additional `advise` contributed $\sim 0.5\%$ wall time — well below criterion noise ($\sim 1\%$). Removing them simplifies `apply_madvise()` to `MADV_SEQUENTIAL + MADV_HUGEPAGE`.

Error contract change. The silent fallback (`if MAP_FAILED { return Self::open(path) }`) was replaced with an explicit `io::ErrorKind::OutOfMemory` error that names the failure and directs the operator to `sudo sysctl vm.nr_hugepages=N`. This makes misconfiguration immediately visible rather than silently degrading to the slower file-mmap path.

References

1. Linux kernel documentation — Huge pages: <https://www.kernel.org/doc/html/latest/admin-guide/mm/hugetlbpage.html>
2. Linux kernel documentation — Transparent Huge Pages: <https://www.kernel.org/doc/html/latest/admin-guide/mm/transhuge.html>
3. `mmap(2)` Linux man page: <https://man7.org/linux/man-pages/man2/mmap.2.html>
4. `madvise(2)` Linux man page: <https://man7.org/linux/man-pages/man2/madvise.2.html>
5. `memfd_create(2)` Linux man page: https://man7.org/linux/man-pages/man2/memfd_create.2.html

6. RFC 8478 — Zstandard Compression and the ‘application/zstd’ Media Type: <https://www.rfc-editor.org/rfc/rfc8478>
 7. Rigtorp, E. — “Latency Implications of Virtual Memory”: <https://rigtorp.se/virtual-memory/>
 8. Rigtorp, E. — “Using Huge Pages on Linux”: <https://rigtorp.se/hugepages/>
 9. Lemire, D. — “Which is fastest: read, fread, ifstream, or mmap?”: <https://lemire.me/blog/2012/06/26/which-is-fastest-read-fread-ifstream-or-mmap/>
 10. LWN — “Relaxed TLB flushes” (2022): <https://lwn.net/Articles/901751/>
 11. arXiv 2409.10946 — “Skip TLB flushes for reused pages within mmmaps”: <https://arxiv.org/abs/2409.10946>
 12. Facebook — zstd streaming API manual: https://facebook.github.io/zstd/zstd_manual.html
 13. Intel — “12th Generation Intel Core Processor Family Datasheet, Vol. 1”: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
 14. Drepper, U. — “What Every Programmer Should Know About Memory”: <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>
-

Appendix A: Build and benchmark reproduction

All benchmarks ran in Rust release mode with the `CARGO_TARGET_DIR` pointed at `/work/cargo-target-ralph` to avoid rebuilding on each worktree switch.

A.1 Synthetic Criterion benchmark

```
# Reserve hugepages (operator action, run once)
sudo sysctl vm.nr_hugepages=160
```

```
# Build and run
CARGO_TARGET_DIR=/work/cargo-target-ralph \
cargo bench --bench decompress -- --save-baseline cycle04
```

Criterion writes HTML reports to `/work/cargo-target-ralph/criterion/decompress/repo`

A.2 CLI real-data benchmark

```
cargo build --release
```

```
export FILE=/mnt/data/Dropbox/split/BINANCE_D/BTCUSD_PERP/20231217.zst
```

```
./target/release/mmapzstd-bench "$FILE" --mode hugepage-memfd --runs 3
./target/release/mmapzstd-bench "$FILE" --mode hugepage-anon --runs 3
./target/release/mmapzstd-bench "$FILE" --mode bufreader --runs 3
```

A.3 smaps inspection

```
# Run while decoding to inspect VMA flags for the mmap region
```

```
CARGO_TARGET_DIR=/work/cargo-target-ralph \
cargo run --release --example measure_mmap_hugepage
```

Relevant /proc/self/smaps fields for the file-mmap VMA on Linux 6.17, ext4:

```
THPEligible:      0
FilePmdMapped:    0 kB
AnonHugePages:    0 kB
VmPTE:            256 kB    (~32,768 PTEs at 8 bytes each)
```

Appendix B: Decoder API reference

The library exposes a single public module, `mmapzstd::decoder`, containing (as of 0.2.0):

Constructor	Platform	On failure	RSS profile
<code>Decoder::from_mmap(path)</code>	all	<code>io::Error</code>	caller-controlled
<code>Decoder::from_slice(data)</code>	all	<code>io::Error</code>	caller-controlled
<code>Decoder::open_hugepage(path)</code>	hugepage	<code>OutOfMemory</code>	full compressed file
<code>Decoder::open_hugepage_memfd(path)</code>	hugepage-memfd	<code>OutOfMemory</code>	full compressed file

`Decoder::open(path)` was removed in 0.2.0; see §9.

All constructors return `io::Result<Decoder>`. `Decoder` implements `std::io::Read`; the caller drives the decode pace. No internal thread is created; no bytes are decompressed ahead of the caller's read calls beyond the zstd frame buffer (maximum 128 KiB per block for standard zstd frames).

The `RETIRE_WINDOW` constant (4 MiB) controls how many compressed bytes behind the decode cursor are retained before `MADV_DONTNEED` is issued. This value exceeds the zstd streaming decoder's compressed-input look-behind

(which is zero: compressed input is consumed monotonically), making the retirement safe in all cases.

Cycle 07 — Streaming Hugepage Decode (S3 Algorithm)

2026-05-26 — mmapzstd 0.3.0

Algorithm

The S3 streaming algorithm introduces `Decoder::open_hugepage_streaming(path, window)`:

- The source file is mmapmed at 4 KiB granularity (no `MAP_POPULATE`, lazy faults).
- A window-byte (default 8 MiB = 4 hugepages) `MAP_HUGETLB | MAP_HUGE_2MB` scratch is allocated once at construction time.
- The decode loop refills the scratch from the source mmap in window-byte chunks, then invokes the zstd streaming decoder exclusively from the hugepage scratch.
- The source mmap is retired (`MADV_DONTNEED`) one scratch-width behind the read cursor.
- Hugepage RSS is bounded to window bytes regardless of corpus size.

Motivation

`open_hugepage_memfd` (the cycle-06 winner at +15% over `BufReader`) requires loading the entire compressed file into hugepage memory. For the i9-12900K test system with $160 \times 2 \text{ MiB} = 320 \text{ MiB}$ of reserved pages, this limits hugepage-backed decoding to files $\leq 320 \text{ MiB}$ compressed. A 1 GiB compressed file requires 514 hugepages — more than the pool. S3 was designed to preserve the dTLB win of H3c while bounding hugepage consumption to a fixed scratch size.

Key bench numbers (cycle 07)

256 MiB decompressed corpus (128.5 MiB compressed), warm cache, Criterion medians:

Mode	Throughput (MB/s)	Hugepage RSS
bufreader-64k	8,912	~0
mmap	8,787	~4 MiB
hugepage-memfd	10,152	~128 MiB

Mode	Throughput (MB/s)	Hugepage RSS
hugepage-streaming-8m	7,791	~8 MiB

1 GiB compressed corpus (1,027.9 MiB compressed → 2,048 MiB), warm cache:

Mode	Throughput (MB/s)	Hugepage RSS	Notes
bufreader-64k	12,128	~0	
mmap	12,843	~4 MiB	
hugepage-memfd	—	—	pool exhausted (needs 514 pages)
hugepage-streaming-8m	10,761	~8 MiB	only hugepage path

Analysis: why streaming is slower

The refill memcpy (source 4 KiB mmap → hugepage scratch) is the dominant cost:

- **256 MiB:** 128.5 MiB compressed ÷ 8 MiB scratch = 16 refills; 128 MiB copied. Each refill scans 8 MiB / 4 KiB = 2,048 4 KiB PTEs, incurring the same TLB pressure as the original mmap-populate path. Over 16 refills: 32,768 PTEs total — identical to the mmap-populate total. Measured overhead: +8 ms vs hugepage-memfd (refill memcpy ≈ 2.5 ms + TLB-miss penalty ≈ 5.5 ms). Streaming is 30.4% slower than hugepage-memfd and 13.4% slower than bufreader-64k on the 256 MiB corpus.
- **1 GiB:** 1,027.9 MiB ÷ 8 MiB ≈ 128 refills; ~1,024 MiB copied. Measured overhead: +22.5 ms vs bufreader. The copy fraction is 11.3% of total decode time at 1 GiB (vs 23.2% at 256 MiB) because total decode time scales with output size while the per-refill copy cost scales with compressed input.

The TLB win from the hugepage scratch is real for the zstd decode step (only 4 PMD entries for the 8 MiB scratch), but the refill copy pays the same 4 KiB PTE tax that originally motivated the hugepage approach, negating the net gain.

Decision — (b) Ship alongside as explicit opt-in

Conditions for (a) not met: streaming is 30% slower than hugepage-memfd on 256 MiB (threshold: $\leq 5\%$).

Conditions for (b) met: - Streaming is $>5\%$ slower than hugepage-memfd on 256 MiB. (yes) - Streaming is the only hugepage-backed path for files exceeding the hugepage pool (1 GiB compressed corpus, pool = 320 MiB). (yes) - The 1 GiB bench completed successfully, bounded RSS at 8 MiB. (yes)

Recommendation: - Files that fit in the hugepage pool: use `open_hugepage_memfd` (fastest, +15% vs `BufReader`). - Files that exceed the hugepage pool: use `open_hugepage_streaming` (only hugepage option; 12.7% below `bufreader`, 16% below `mmap` at 1 GiB, but bounded 8 MiB hugepage RSS). - No-hugepage workloads: `from_mmap` (portable, ~ 4 MiB RSS, within 5% of `BufReader` at 1 GiB).

Surprises

mmap beats streaming and BufReader at 1 GiB. At 256 MiB, `mmap` trails `BufReader` by 1.4%; at 1 GiB, `mmap` is 5.7% faster than `BufReader` (12,843 vs 12,128 MB/s). The zero-copy advantage of `mmap` over `BufReader`'s `read()` syscalls grows with corpus size; the TLB penalty (proportional to PTEs) remains constant per MB and becomes a smaller fraction of total time as the decode workload grows.

Streaming overhead percentage shrinks at 1 GiB. The refill's TLB penalty per refill is fixed ($8 \text{ MiB} / 4 \text{ KiB} = 2,048$ PTEs); the decode work per refill scales with how many decompressed bytes `zstd` produces from one scratch fill. On the 2:1 corpus, each 8 MiB compressed chunk decompresses to ~ 16 MiB; at higher compression ratios the decode work per refill grows, further amortising the refill overhead.

A larger window would help. §5 of `docs/streaming-bench.md` notes that a 64 MiB window would reduce refill count $8\times$ (2 refills for 256 MiB), potentially closing the gap to $\leq 5\%$ of `BufReader`. This is left as future work; it requires 32 hugepages (64 MiB) reserved, which is well within the 160-page pool.

Decisions for future cycles

- The primary bottleneck is the refill `memcpy` from 4 KiB source pages — not the decoder itself. A larger window is the clearest lever; experiment with 64 MiB.
- The streaming path is the only one that enables hugepage-backed decode of files larger than the hugepage pool; this is its unique value even when slower than `mmap`.

- `DEFAULT_STREAMING_WINDOW = 8 MiB` (4 hugepages) is a reasonable default for constrained hugepage pools; document it as the minimum viable window.
- The 50/50 random+repetitive corpus at 2:1 compression is a poor proxy for compression-ratio sensitivity (see cycle-06 distillation). A structured corpus with higher compressible fraction would let future cycles measure the per-refill decode-work amortisation more cleanly.